Programmation orientée objet (UAA14)

Exercices : série 2

Objets d'apprentissage :

- ✔ Modéliser une logique de programmation orientée objet.
- ✔ Déclarer une classe.
- ✔ Instancier une classe.
- ✔ Utiliser les méthodes de l'objet instancié.
- ✔ Traduire un algorithme dans un langage de programmation.
- ✔ Commenter des lignes de code.
- ✔ Tester le programme conçu.
- ✔ Caractériser les attributs dans une classe (encapsulation).
- ✔ Caractériser les méthodes dans une classe (encapsulation).
- ✔ Décrire la création d'un constructeur.
- ✔ Extraire d'un cahier des charges les informations nécessaires à la programmation.
- ✔ Programmer en recourant aux instructions et types de données nécessaires au développement d'une application.
- ✔ Corriger un programme défaillant.
- ✔ Développer une classe sur la base d'un cahier des charges en respectant le paradigme de la programmation orientée objet.
- ✔ Programmer en recourant aux classes nécessaires au développement d'une application orientée objet.
- ✔ Améliorer un programme pour répondre à un besoin défini.

Exercice 1 : modéliser avant de coder

Jusqu'ici, on t'a toujours dit quelle classe écrire et quelles variables d'instance lui donner. Dans un vrai projet, personne ne te le dira : c'est le premier travail du programmeur, et il se fait **avant** d'écrire la moindre ligne de code.

Cet exercice ne demande aucun code. Réponds sur feuille ou dans un document.

Etape 1

Lis attentivement le cahier des charges suivant.

```
Un club de tennis souhaite informatiser la réservation de ses terrains.

Le club dispose de plusieurs terrains. Chaque terrain porte un numéro,
possède une surface (terre battue, gazon ou synthétique) et se trouve soit
à l'intérieur, soit à l'extérieur. Le club veut pouvoir afficher un libellé
complet du terrain, du genre « Terrain 3 – gazon (extérieur) ».

Chaque membre du club est identifié par son nom, son prénom et sa date de
naissance. Le club veut pouvoir afficher l'âge d'un membre et savoir si ce
membre est mineur.

Un membre réserve un terrain pour une date, une heure de début et une durée
exprimée en minutes. Le club veut connaître l'heure de fin d'une
réservation ainsi que son prix : 12 euros de l'heure, réduit de moitié pour
les membres mineurs.
```

Etape 2

Repère les classes. Pour chacune, donne son nom et justifie en une phrase pourquoi elle mérite d'être une classe.

Etape 3

Pour chaque classe, liste ses variables d'instance et donne le type de chacune (chaine, entier, réel, booléen). Pour une date ou une heure, une chaine de caractères fera l'affaire.

Attention : toutes les informations du texte ne sont pas des variables d'instance.

Etape 4

Relis ta liste. Pour chaque information que tu y as notée, pose-toi la question : est-elle **donnée**, ou peut-elle se **calculer** à partir des autres ?

Déplace dans une seconde liste, intitulée « méthodes », tout ce qui est calculable. Justifie chaque déplacement en une phrase.

Etape 5

Compare ta modélisation avec celle d'un camarade. Vous n'avez probablement ni les mêmes classes, ni les mêmes méthodes.

Pour chaque différence, demandez-vous laquelle des deux versions serait la plus simple à faire évoluer si le club décidait, demain, de louer aussi des raquettes.

Exercice 2 : dépannage

Le programme ci-dessous devrait afficher l'état de lecture d'un livre… mais il ne fonctionne pas.

Etape 1

Recopie ce programme dans la sandbox et exécute-le.

```php
<?php
class Livre {

    private $titre;
    private $auteur;
    private $nbPages;
    private $nbPagesLues;

    public function __construct($titre, $auteur, $nbPages) {
        $titre = $titre;
        $this->auteur = $auteur;
        $this->nbPages = $nbPages;
        $this->nbPagesLues = 0;
    }

    public function lirePages($nombre) {
        $this->nbPagesLues += $nombre;
    }

    public function pagesRestantes() {
        $restantes = $nbPages - $this->nbPagesLues;
        return $restantes;
    }

    public function afficherEtat() {
        echo "Livre : " . $this->titre . "\n";
        echo "Pages lues : " . $this->nbPagesLues . "\n";
        echo "Pages restantes : " . $restantes . "\n";
    }
}

$monLivre = new Livre("Le PHP pour les nuls", "R. Dupont", 300);
$monLivre->lirePages(75);
echo "Il reste " . $monLivre->pagesRestantes() . " pages.\n";
$monLivre->afficherEtat();
```

Conseil : pense à activer l'affichage des erreurs (voir l'exercice 2 de la série 1), sans quoi PHP restera muet.

Etape 2

Repère les trois erreurs. Pour chacune, écris **en commentaire dans le code** pourquoi PHP ne trouve pas, à cet endroit, la valeur attendue.

Attention : ces erreurs ne provoquent pas toutes un message. Certaines se contentent de produire un résultat vide ou faux.

Etape 3

Corrige les erreurs une par une, en réexécutant le programme après chaque correction. Au final, il doit afficher :

```
Il reste 225 pages.
Livre : Le PHP pour les nuls
Pages lues : 75
Pages restantes : 225
```

Etape 4

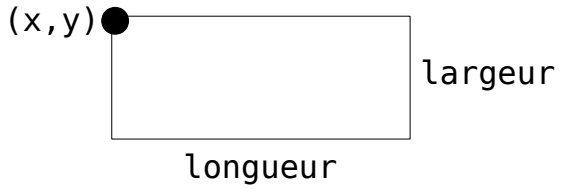
Pour terminer, réponds aux trois questions suivantes :

- Une variable d'instance est-elle accessible dans toutes les méthodes de sa classe ? Comment y accède-t-on ?
- Une variable déclarée à l'intérieur d'une méthode est-elle accessible dans une autre méthode de la même classe ? Pourquoi ?
- Que se passe-t-il lorsqu'un argument porte le même nom qu'une variable d'instance ?

Exercice 3 : rectangle

Etape 1

Crée une classe intitulée `Rectangle` qui doit permettre de créer des rectangles. Un rectangle est caractérisé par des coordonnées x et y (la position du coin supérieur gauche), une largeur et une longueur.



La classe `Rectangle` contient les variables d'instances suivantes :

- `$x` : coordonnée X du point supérieur gauche
- `$y` : coordonnée Y du point supérieur gauche
- `$largeur` : la largeur du rectangle
- `$longueur` : la longueur du rectangle

Etape 2

Crée un constructeur pour la classe `Rectangle` (constructeur à 4 arguments).

Etape 3

Fais en sorte que les v.i `largeur` et `longueur` puissent être accédées en lecture et en écriture depuis l'extérieur de la classe.

Ajoute également les méthodes suivantes à la classe `Rectangle` :

- `périmètre` : retourne le périmètre (calculé) d'un rectangle
- `surface` : retourne la surface (calculée) du rectangle
- `déplacer` : modifie l'emplacement du coin supérieur gauche en une nouvelle position `(x,y)` sur base de deux entiers reçus en argument

Etape 4

Dans la sandbox, effectue les opérations suivantes (dans l'ordre) :

1. crée un objet de type `Rectangle` avec les propriétés de ton choix
2. affiche à l'écran les coordonnées X et Y du coin supérieur gauche du rectangle créé ainsi que la largeur et la longueur de ce rectangle.
3. modifie la longueur et la largeur de ce rectangle
4. affiche à nouveau à l'écran la largeur et la longueur du rectangle
5. crée d'autres rectangles avec d'autres valeurs et fais appel aux différentes méthodes de la classe

Exercice 4 : entrainement

Etape 1

Crée une classe nommée `Entrainement` permettant de créer des entrainements de course à pied.

Un entrainement est caractérisé par :

- le nom du coureur ;
- la longueur (en mètres) du parcours course à pied ;
- le temps (en secondes) mis par le coureur pour le parcourir.

Crée les variables d'instance nécessaires avec les types adéquats.

Etape 2

Crée un constructeur pour la classe `Entrainement`.

Etape 3

Ajoute les méthodes suivantes :

- `vitesseMoyenne` : retourne la vitesse moyenne (en km/h) du coureur (attention, une valeur réelle et pas entière !)
- `comparerTemps` : reçoit un entier en argument représentant le temps en secondes mis lors d'un autre entrainement et retourne la différence entre ce temps et le temps mis par le coureur lors de l'entrainement courant

Etape 4

Dans la sandbox, effectue les opérations suivantes :

1. crée un objet de type `Entrainement`
2. affiche à l'écran le nom du coureur réalisant cet entrainement
3. affiche à l'écran (en faisant appel aux méthodes adéquates) la vitesse moyenne lors de l'entrainement et la comparaison entre le temps de cet entrainement et le temps d'un autre (de ton choix)

Exercice 5 : placement

Etape 1

Crée la classe nommée `Placement` permettant de gérer des capitaux placés pendant un certains temps dans une banque.

Un placement est caractérisé par :

- un nombre total d'années durant lequel le capital est placé (ex. 10 ans)
- un taux d'intérêts (ex. 2,5)
- un montant placé

Etape 2

Prévois un constructeur permettant de créer un placement.

Etape 3

Ajoute les méthodes suivantes :

- `déterminerCapitalPartiel` : reçoit en argument un nombre d'années et calcule la valeur du capital après ce nombre d'années.
  La formule à employer est : $montant * (1 + taux/100)^{durée}$
- `déterminerCapitalFinal` : calcule la valeur finale du montant placé (utilise la méthode précédente !)
- `déterminerCapitalAcquis` : calcule le montant que le client va percevoir sachant que l'état perçoit un certain pourcentage (reçu en argument)

Etape 4

Dans la sandbox PHP :

- construire un objet de type `Placement` : le montant de départ est de 1000€ avec une durée de 10 ans et un taux de 2,5%
- afficher à l'écran le capital partiel après 3 ans, le capital final et le montant perçu de ce placement (l'état perçoit 10% de la somme)

Exercice 6 : une collection d'objets

Jusqu'ici, tu as créé tes objets un par un, chacun dans sa propre variable. Dès qu'une application en manipule des dizaines, cette façon de faire ne tient plus : on range les objets dans un **tableau**.

Etape 1

Crée une classe `Chanson` caractérisée par :

- un titre ;
- un artiste ;
- une durée, exprimée en secondes.

Prévois un constructeur et un `__get` générique (comme à l'exercice 3 de la série 1).

Etape 2

Ajoute à la classe `Chanson` une méthode `dureeFormatee` qui retourne la durée sous la forme « 3:45 » (les minutes, deux-points, les secondes). Attention : les secondes doivent toujours s'afficher sur deux chiffres, donc « 4:05 » et non « 4:5 ».

Etape 3

Dans la sandbox, crée un tableau vide puis remplis-le de cinq chansons :

```php
$playlist = array();

array_push($playlist, new Chanson("Bohemian Rhapsody", "Queen", 355));
array_push($playlist, new Chanson("Numb", "Linkin Park", 187));
// ... et trois autres de ton choix
```

Un tableau peut contenir des objets exactement comme il contient des entiers ou des chaines de caractères.

Etape 4

Parcours la playlist et affiche, pour chaque chanson, son titre, son artiste et sa durée formatée.

```php
foreach ($playlist as $chanson) {
    echo $chanson->titre . " ...";
}
```

Etape 5

Toujours dans la sandbox, écris le code qui affiche :

- la durée totale de la playlist, elle aussi au format minutes:secondes ;
- le titre de la chanson la plus longue ;
- le nombre de chansons d'un artiste donné.

Etape 6

Tu viens d'écrire ces trois calculs dans la sandbox, et non dans la classe `Chanson`. Explique pourquoi ils ne peuvent pas être des méthodes de cette classe.

Autrement dit : de quelle information une méthode de `Chanson` ne dispose-t-elle pas, alors qu'elle serait indispensable pour calculer la durée totale de la playlist ?

Exercice 7 : véhicules

Etape 1

Crée une classe nommée `Véhicule` permettant de créer des véhicules. Un véhicule est caractérisé par :

- une plaque d'immatriculation (peut contenir des chiffres, des lettres et des tirets) ;
- une marque ;
- un modèle ;
- une capacité maximale du réservoir ;
- un niveau de jauge de carburant (nombre de litres de carburant réellement dans le réservoir).

Etape 2

Crée un constructeur pour la classe `Véhicule`.

Etape 3

Adapte le constructeur afin de faciliter la création de véhicules avec un réservoir plein. Autrement dis, il n'est pas nécessaire de préciser le niveau de la jauge de carburant lors de la création d'un véhicule : ce niveau est automatiquement égal à la capacité maximale du réservoir.

Etape 4

Ajoute les méthodes suivantes :

- `typeVéhicule` : retourne une chaine de caractère contenant la marque et le modèle du véhicule (séparés par un espace)
- `ajouterCarburant` : reçoit en argument une quantité (en litres) de carburant et rajoute cette quantité dans le réservoir (attention à ne pas déborder !)
- `état` : retourne la chaine de caractère suivante :
  « `<type>` (`<jauge>` / `<capacité max>` litres) » (ex. « Opel Adam (20/30 litres) »)

Etape 5

Adapte la méthode `ajouterCarburant` afin que la quantité soit facultative : lorsqu'aucune quantité n'est précisée, le réservoir est rempli au maximum.

Etape 6

Dans la sandbox, réalise les opérations suivantes :

1. crée plusieurs véhicules de ton choix (en utilisant également les valeurs par défaut)
2. affiche l'état de tous les véhicules créés (en utilisant la méthode `état`)
3. teste la méthode `ajouterCarburant` en l'appelant de différente manière (avec / sans la valeur par défaut)
4. ré-affiche l'état de tous les véhicules et vérifie le bon fonctionnement de la méthodes `ajouterCarburant`

Exercice 8 : commande

Etape 1

Crée une classe `Commande` permettant de gérer des commandes d'articles effectués par des clients.

Par facilité, une commande ne peut contenir qu'un seul article mais ce dernier peut être commandé en plusieurs exemplaires (ex. Une commande comporte 3 exemplaires du livre « Le PHP pour les nuls »).

Une commande est caractérisée par :

- le nom du client qui passe la commande ;
- le libellé de l'article commandé ;
- le nombre d'exemplaires de l'article commandé ;
- le prix unitaire (en €) de l'article commandé.

Etape 2

Crée un constructeur permettant de créer une commande. Fais en sorte que le constructeur permette de créer facilement des commandes avec un seul exemplaire.

Etape 3

Ajoute les méthodes suivantes :

- `modifierQuantité` : permet de modifier la quantité commandée
- `coûtTotal` : calcule le coût total de la commande
- `imprimerBonCommande` : affiche la commande sous la forme suivante

```
BON DE COMMANDE DU CLIENT <nom>

Article : <libellé>
Quantité commandée : <quantité>
Prix unitaire : <prix> euros

COUT TOTAL : <total> EUROS
```

Etape 4

Dans la sandbox, crée des commandes et affiche le bon de commande pour chacune d'elles.

Exercice 9 : le distributeur

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des variables d'instance, ni celle des méthodes : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

Un distributeur automatique propose plusieurs boissons. Chaque boisson porte un nom, un prix en euros et occupe un emplacement contenant un certain nombre de canettes.

Un emplacement a une capacité de 12 canettes et, à la mise en service, il est plein : on ne précise donc pas le stock au moment de la création. Le distributeur doit pouvoir vendre une canette, être réapprovisionné — à défaut d'indication, jusqu'à la capacité — et afficher son inventaire complet.

Les exigences

- la valeur en euros du stock d'une boisson (prix × nombre de canettes) ne peut pas être une variable d'instance ;
- vendre une canette alors que l'emplacement est vide ne doit rien casser ;
- un réapprovisionnement sans argument remplit jusqu'à la capacité ;
- le calcul de la valeur totale du distributeur ne peut pas être une méthode de ta classe : explique pourquoi dans un commentaire.

Le résultat attendu

Ton programme de test crée au moins quatre boissons, les range dans un tableau, effectue quelques ventes et réapprovisionnements, puis affiche l'inventaire. La mise en forme exacte est libre ; les chiffres dépendent de tes essais.

```
--- INVENTAIRE ---
Eau plate      0.80 EUR  x 12  =   9.60 EUR
Cola           1.50 EUR  x  9  =  13.50 EUR
Jus d'orange   1.80 EUR  x 12  =  21.60 EUR
Cafe           1.20 EUR  x  0  =   0.00 EUR   [EPUISE]
------------------
Valeur totale du stock : 44.70 EUR
```

Pour vérifier

- quelles informations as-tu choisi de calculer plutôt que de stocker ? pourquoi ?
- pourquoi la valeur par défaut du réapprovisionnement ne peut-elle pas s'écrire directement dans la signature de la méthode ?
- si les emplacements passaient de 12 à 20 canettes, combien d'endroits devrais-tu modifier dans ton programme ? et combien devrais-tu en modifier idéalement ?

Exercice 10 : la médiathèque

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des variables d'instance, ni celle des méthodes : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

La médiathèque de l'école prête des livres. Chaque livre porte un titre, un auteur, et existe en un certain nombre d'exemplaires.

La plupart des livres n'existent qu'en un seul exemplaire : sauf indication contraire, un livre ajouté au catalogue en compte donc un. Au moment où il est ajouté, aucun exemplaire n'est sorti. La médiathèque doit pouvoir prêter un exemplaire, en récupérer — à défaut d'indication, tous les exemplaires sortis, comme lors de l'inventaire de fin d'année —, acheter des exemplaires supplémentaires — un seul, sauf indication contraire — et afficher l'état de ses prêts.

Les exigences

- le nombre d'exemplaires disponibles et le pourcentage d'exemplaires sortis ne peuvent pas être des variables d'instance ;
- prêter un livre dont tous les exemplaires sont sortis ne doit rien casser, et le programme de test doit pouvoir savoir que le prêt a été refusé, pour l'afficher ;
- récupérer plus d'exemplaires qu'il n'en est sorti ne doit rien casser : le nombre d'exemplaires sortis ne devient jamais négatif ;
- la recherche du livre le plus demandé et le nombre total d'exemplaires sortis ne peuvent pas être des méthodes de ta classe : explique pourquoi dans un commentaire.

Le résultat attendu

Ton programme de test crée au moins quatre livres, les range dans un tableau, effectue des prêts (dont au moins un refusé), un achat et des retours, puis affiche l'état des prêts. La mise en forme exacte est libre ; les chiffres dépendent de tes essais.

```
Pret de Harry Potter 1 : refuse, aucun exemplaire disponible

--- MEDIATHEQUE ---
Le Petit Prince   Saint-Exupery  3 ex.  2 sortis  1 dispo  ( 67 %)
Harry Potter 1    Rowling        5 ex.  5 sortis  0 dispo  (100 %)  [TOUS SORTIS]
L'Etranger        Camus          2 ex.  1 sortis  1 dispo  ( 50 %)
Bel-Ami           Maupassant     1 ex.  0 sortis  1 dispo  (  0 %)
------------------
Livre le plus demande : Harry Potter 1 (100 %)
Exemplaires sortis : 8 sur 11
```

Pour vérifier

- quelles informations as-tu choisi de calculer plutôt que de stocker ? pourquoi ?

- ta classe utilise plusieurs valeurs par défaut : lesquelles s'écrivent directement dans la signature, et laquelle ne le peut pas ? pourquoi ?
- comment ta méthode de prêt fait-elle savoir au programme de test que le prêt a été refusé ?
- pourquoi la recherche du livre le plus demandé ne peut-elle pas être une méthode de ta classe ?

Exercice 11 : le voyage scolaire

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des variables d'instance, ni celle des méthodes : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

Pour le voyage de fin d'année à Rome, chaque élève paie sa participation en plusieurs versements. Le voyage coûte 350 euros ; certains élèves bénéficient toutefois d'un prix réduit, fixé au moment de leur inscription. Un élève qui vient de s'inscrire n'a encore rien versé.

On doit pouvoir enregistrer un versement — sans montant précisé, l'élève paie d'un coup tout ce qu'il lui reste à payer —, enregistrer le désistement d'un élève, qui récupère alors tout ce qu'il a versé, et afficher le suivi des paiements.

Les exigences

- le reste à payer et le pourcentage déjà payé ne peuvent pas être des variables d'instance ;
- un versement plus élevé que le reste à payer n'est encaissé qu'à hauteur de ce reste, et le programme de test doit pouvoir afficher le montant réellement encaissé ;
- un élève qui s'est désisté ne doit plus rien, et n'est plus compté parmi les participants ;
- le prix normal du voyage (350 euros) n'apparaît qu'à un seul endroit de ton programme ;
- le total récolté, le montant qu'il reste à récolter et le nombre d'élèves en ordre de paiement ne peuvent pas être des méthodes de ta classe.

Le résultat attendu

Ton programme de test inscrit au moins quatre élèves, dont un au prix réduit, les range dans un tableau, enregistre des versements (dont un trop élevé et un sans montant) et un désistement, puis affiche le suivi. La mise en forme exacte est libre ; les chiffres dépendent de tes essais.

```
Versement de 150.00 EUR pour Zoe : 150.00 EUR encaisses
Versement de 120.00 EUR pour Malik : 120.00 EUR encaisses
Versement de 50.00 EUR pour Sam : 50.00 EUR encaisses
Versement de 100.00 EUR pour Lea : 100.00 EUR encaisses
Versement de 250.00 EUR pour Zoe : 200.00 EUR encaisses
Desistement de Lea : 100.00 EUR rembourses
Sam solde son voyage : 150.00 EUR encaisses
Versement de 50.00 EUR pour Malik : 50.00 EUR encaisses

--- VOYAGE A ROME ---
Zoe    350.00 EUR  verse 350.00  reste   0.00  (100 %)  [EN ORDRE]
Malik  350.00 EUR  verse 170.00  reste 180.00  ( 49 %)
Sam    200.00 EUR  verse 200.00  reste   0.00  (100 %)  [EN ORDRE]
Lea    350.00 EUR                          [DESISTEMENT]
----------------------
Total recolte    : 720.00 EUR
```

```
Reste a recolter : 180.00 EUR
Eleves en ordre  : 2 sur 3 participants
```

Pour vérifier

- quelles informations stockes-tu, et lesquelles calcules-tu ? pourquoi ?
- le prix du voyage et le montant d'un versement ont tous les deux une valeur par défaut : laquelle peut s'écrire directement dans la signature, et laquelle ne le peut pas ? pourquoi ?
- que retourne ta méthode de versement, et pourquoi ce choix ?
- un élève désisté a-t-il encore un « reste à payer » ? comment ton programme en tient-il compte dans le reste à récolter ?
- si le voyage passait à 380 euros, combien de lignes devrais-tu modifier ?

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/introduction-programmation-orientee-objet/
- Cours de « Développement d'applications WEB », Hénallux (2019)